Agent State

- messages: list[all messages]
- url: string
- extraction_duration_seconds: float
- structured_dom_chars: int
- structured_dom_info: list[dict]
- cmp_type: string
- settings_extracted: bool
- batch_mode: bool
- llm_calls: int
- model_aborted: bool
- abort_reason: str
- suspected_stuck_reason: str
- human_review_count: int
- current_rule_draft: string
- last_test_result: dict
- test_rule_count: int
- screenshot_info: dict
- analyze_screenshot_count: int
- error_history: list[str]
- last_error: string
- final_result: dict

Persistence: SqliteSaver
(State survives program restart)

START

extraction_node:
Fetches and pre-processes DOM, identifies CMP type if possible

llm_node:
Receives DOM data, last messages and system prompt with injected static few-shot examples; selects next action

budget = 20 + 5 * human_review_count

llm_calls >= budget?

Yes

No

Did the LLM call a tool?

No

Yes

Observation:
loop back

human_review_node:
→ interrupt() pauses agent
→ State persisted in SQLite
→ Waiting for external feedback
→ completed review increments human_review_count by 1
→ thus extends LLM-call budget by 5

external input:
Feedback via console

LLM called request_human_review

tool_node:
Tool is executed
- analyze_screenshot
- test_rule (including schema validation using Pydantic)
- request_human_review

Tool result:
loop back

rule_output_node:
Parses the final LLM message for terminal tags

Abort tags found?

Yes

Store abort_reason & set model_aborted = True

No

Rule found and deserializable to a dictionary?

No

Yes

Rule gets stored in final_result

END